

# Technical Challenge: MCP HTTP Server + LangGraph ReAct Agent

## Goal

Build a Python MCP HTTP server that exposes 3 tools, and a Python LLM agent (ReAct-style in LangGraph) that consumes the MCP to answer the question:

*“What is the weather forecast of the data center?”*

For this challenge, “data center” means: *the machine running the MCP server.*

The agent must obtain the data center’s forecast by calling MCP tools in this exact order:

1. IPify -> To get the public IP of the machine running the MCP server
2. IP-API -> To get the latitude/longitude of the IP previously gotten
3. Open-Meteo -> To fetch the weather forecast for those coordinates

No front-end is required. The interaction must happen in the CMD terminal.

## Tools to integrate

Your MCP server must integrate the following public APIs:

1. **Weather Forecast (Open-Meteo)**

Docs: <https://open-meteo.com/en/docs>

Purpose: Get weather forecast given latitude/longitude.

2. **IPify**

Docs: <https://www.ipify.org/>

Purpose: Get the public IP of the device running the MCP server.

3. **IP-API**

Example: <http://ip-api.com/json/{ip}>

Purpose: Obtain the latitude/longitude of a given IP.

## LLM requirement

Use free LLM model such as the Gemini API or any other you could have access to.

Docs: <https://ai.google.dev/gemini-api/docs/pricing>

## Deliverables

Submit a repository containing:

### 1) MCP HTTP Server (Python)

- Runs locally over **HTTP**
- Exposes **exactly three tools** (ipify, ip\_to\_geo, weather\_forecast).
- Includes:
  - input validation
  - clear error responses
  - structured logs

### 2) MCP Client (built into the agent)

- A small Python client used by the agent to:
  - fetch the tool list
  - call tools reliably
  - validate responses

### 3) LLM Agent (LangGraph, ReAct)

- A console program that:
  - Accepts user question(s) via terminal
  - Uses a **ReAct** loop implemented with LangGraph
  - Calls the MCP tools in the required sequence
  - Outputs the final answer in plain text + a short “trace” of tool calls

### 4) Documentation (important)

We will heavily evaluate documentation. Include:

- architecture overview
- explanation of your LangGraph state, nodes, and tool-calling strategy
- assumptions/limitations